

03. Generators

Ek line me

Aisa function jo ek ke baad ek kai values de sakta hai, jab zarurat ho tab (pause aur resume ke saath).

Syntax

```
js

function* generateSequence() {
  yield 1;
  yield 2;
  return 3;
}
```

- `function*` (star ke saath) = generator function.
- Call karne par code **run nahi hota**, ek *generator object* milta hai.

`next()` method

Har `next()` call agle `yield` tak code chalata hai aur ek object deta hai:

```
js

let g = generateSequence();
g.next(); // {value: 1, done: false}
g.next(); // {value: 2, done: false}
g.next(); // {value: 3, done: true}
```

- `value`: yield ki hui value
- `done`: `true` matlab function khatam

Generators iterable hote hain

```
js

for (let v of generateSequence()) console.log(v); // 1, 2
```

Dhyan do: `for..of` last `return` wali value (3) ko **ignore** karta hai. Sab dikhana ho to `yield` use karo. Spread bhi chalta hai: `[0, ...generateSequence()]`.

Iterable banana aasan

```
js

let range = {
  from: 1, to: 5,
  *[Symbol.iterator]() {
    for (let v = this.from; v <= this.to; v++) yield v;
  }
};

[...range]; // 1,2,3,4,5
```

Generator Composition: `yield*`

Ek generator ke andar doosra generator embed karna:

```
js

function* nums(s, e) { for (let i = s; i <= e; i++) yield i; }
function* all() {
  yield* nums(48, 57);
  yield* nums(65, 90);
}
```

`yield` do-tarfa hai

`next(value)` se andar value bhej sakte ho, wo `yield` ka result ban jaati hai:

```
js

function* gen() {
  let ans = yield "2 + 2 = ?";
  console.log(ans); // 4
}

let g = gen();
g.next(); // question milta hai
g.next(4); // answer andar gaya
```

(Pehla `next()` bina argument ke hi call karo.)

`throw` aur `return`

- `generator.throw(err)`: generator ke andar us `yield` wali line par error throw karta hai.
- `generator.return(value)`: generator ko turant khatam karke value deta hai.

Yaad rakho

- Generators aajkal kam use hote hain, par iterables banane aur data streams ke liye kaafi useful hain.
- Infinite generator bana sakte ho, bas loop me `break` zaroor lagao.